



What is Containerization?

- **Containerization** – is a form of virtualization where ***applications run in isolated*** user spaces, called containers, while **using the same shared operating system** (OS). One of the benefits of containerization is that a container is essentially a fully packaged and **portable computing environment**.
- Containers encapsulate an application as a single executable package of software that bundles application code together with all of the related configuration files, libraries, and dependencies required for it to run. Containerized applications are “isolated” in that they do **not bundle in a copy of the operating system**.

What is Containerization?

- With containerization technology, there's **less overhead during startup** and no need to set up a separate guest OS for each application since they all share the same OS kernel. Because of this high efficiency, containerization is commonly used for packaging up the many individual **microservices** that make up modern apps.

Why Containerization?

- **Containerization** allows developers to create and deploy applications faster and more securely. With traditional methods, code is developed in a specific computing environment which, when **transferred to a new location**, often results in bugs and errors. For example, when a developer transfers code from a desktop computer to a virtual machine (VM) or from a Linux to a Windows operating system. Containerization eliminates this problem by bundling the application code together with the related configuration files, libraries, and dependencies required for it to run. This single package of software or “container” is abstracted away from the host operating system, and hence, it stands alone and becomes portable, able to run **across any platform or cloud**, free of issues.

The Benefit of Containerization?

- Portability: A container creates an **executable package** of software that is abstracted away from (not tied to or dependent upon) the host operating system, and hence, is portable and able to **run uniformly and consistently across any platform or cloud**.
- Speed: Containers are often referred to as “lightweight,” meaning they **share the machine’s operating system (OS) kernel** and are not bogged down with this extra overhead. Not only does this drive higher server efficiencies, it also reduces server and licensing costs while **speeding up start-times** as there is no operating system to boot.

The Benefit of Containerization?

- **Fault isolation:** Each containerized application is isolated and operates independently of others. The failure of one container does **not affect the continued operation of any other containers**. Development teams can identify and correct any technical issues within one container without any downtime in other containers. Also, the container engine can leverage any OS **security isolation techniques**—such as SELinux access control—to isolate faults within containers.
- **Efficiency:** Software running in containerized environments shares the machine's OS kernel, and application layers within a container can be shared across containers. Thus, containers are inherently **smaller in capacity than a VM and require less start-up time**, allowing far more containers to run on the same compute capacity as a single VM. This drives higher server efficiencies, reducing server and licensing costs.

The Benefit of Containerization?

- **Easy to manage:** A container orchestration platform automates the installation, scaling, and management of containerized workloads and services. Container orchestration platforms can ease management tasks such as **scaling containerized apps, rolling out new versions of apps, and providing monitoring, logging and debugging, among other functions.** Kubernetes, perhaps the most popular container orchestration system available, is an open source technology (originally open-sourced by Google, based on their internal project called Borg) that automates Linux container functions originally. Kubernetes works with many container engines, such as Docker, but it also works with any container system that conforms to the Open Container Initiative (OCI) standards for container image formats and runtimes.

The Benefit of Containerization?

- **Security:** The isolation of applications as containers inherently prevents the invasion of malicious code from affecting other containers or the host system. Additionally, security permissions can be defined to automatically block unwanted components from entering containers or limit communications with unnecessary resources.

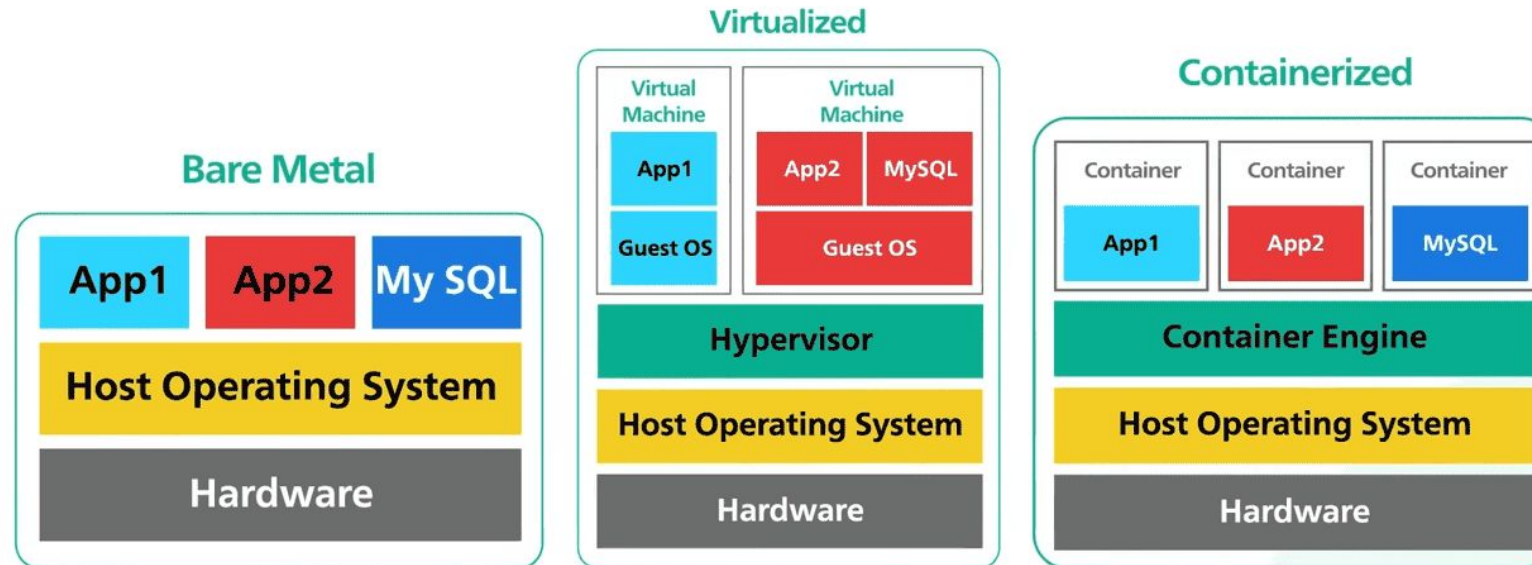
Containerization vs VM

Virtual Machine	Docker Container
Hardware-level process isolation	OS level process isolation
Each VM has a separate OS	Each container can share OS
Boots in minutes	Boots in seconds
VMs are of few GBs	Containers are lightweight (KBs/MBs)
Ready-made VMs are difficult to find	Pre-built docker containers are easily available
VMs can move to new host easily	Containers are destroyed and re-created rather than moving
Creating VM takes a relatively longer time	Containers can be created in seconds
More resource usage	Less resource usage

Bare Metal , Virtualized , Containerized

Bare metal, VMs, or Containers

ByteByteGo.com



What is Docker?

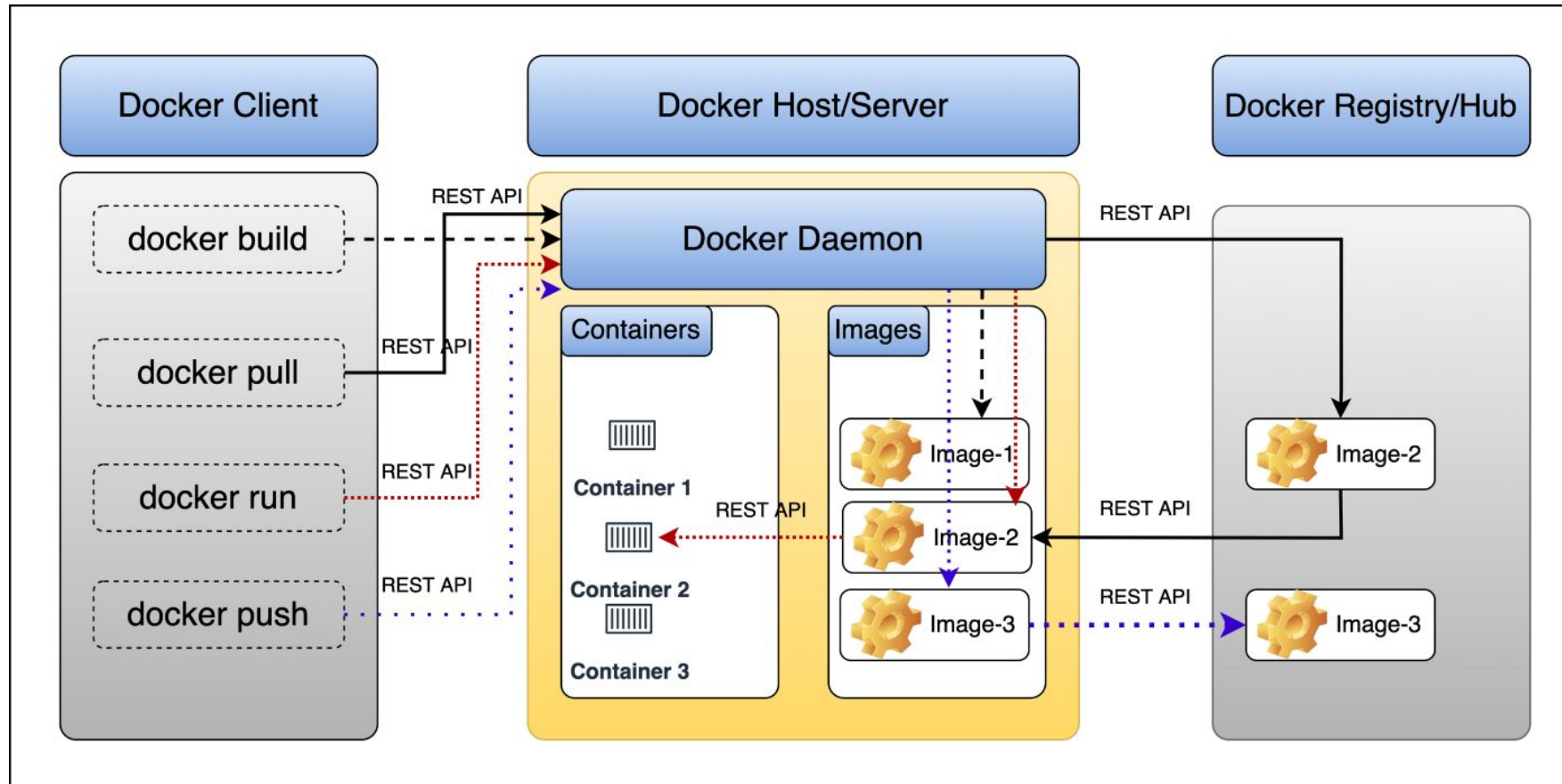
- Docker is an open platform for **developing, shipping, and running applications**.
Docker enables you to **separate your applications** from your infrastructure so you can deliver software quickly.
- With Docker, you can manage your infrastructure in the same ways you manage your applications. By taking advantage of Docker's methodologies for shipping, testing, and deploying code quickly, you can significantly **reduce the delay between writing code** and running it in production.

What can we do with Docker?

- Docker allows User to:
 - Delivery the applications to end-user quickly : Docker provides the ability to package and run an application in a **loosely isolated environment** called a container.
 - High Availability of Data: Docker enable user to **backup data** from its container which we called docker-volume.
 - High Availability of Your Application: Docker give the **scalable feature to prevent the website from down time** due to busy traffic to the website...

Docker Architecture

- Docker uses a client-server architecture.



Docker Architecture

- The **Docker client** (docker) is the agent to interact with **Docker Daemon**. When you use commands such as docker run, the client sends these commands to **dockerd** to perform what you want docker do. The docker command uses the **Docker API**.
- The Docker daemon (**dockerd**) **listens for Docker API requests** and manages Docker objects such as images, containers, networks, and volumes.
- A **Docker registry** stores Docker images. **Docker Hub** is a public registry that anyone can use, and Docker is configured to look for images on Docker Hub by default. You can even run your own **private registry**. When you use the docker pull or docker run commands, the required images are pulled from your configured registry. When you use the docker push command, your image is pushed to your configured registry.
- **Docker objects**- When you use Docker, you are creating and using **images, containers, networks, volumes**, plugins, and other objects. This section is a brief overview of some of those objects.

Docker Command

- **Installation:** <https://docs.docker.com/engine/install/ubuntu/>
 - `$ curl -fsSL https://get.docker.com -o get-docker.sh`
 - `$ sudo sh get-docker.sh`
 - `$ sudo systemctl start docker`
 - `$ sudo systemctl status docker`
 - `$ sudo systemctl enable docker`
 - `$ sudo usermod -aG docker $(whoami)` # Then Logout current user
 - `$ sudo docker -v` # Show version
 - `$ sudo docker ps -a` # Show all stopped and running container
 - `$ sudo docker ps -l` # Show the last container
 - `$ sudo docker ps -n3` # Show the last 3 lines

Docker Command

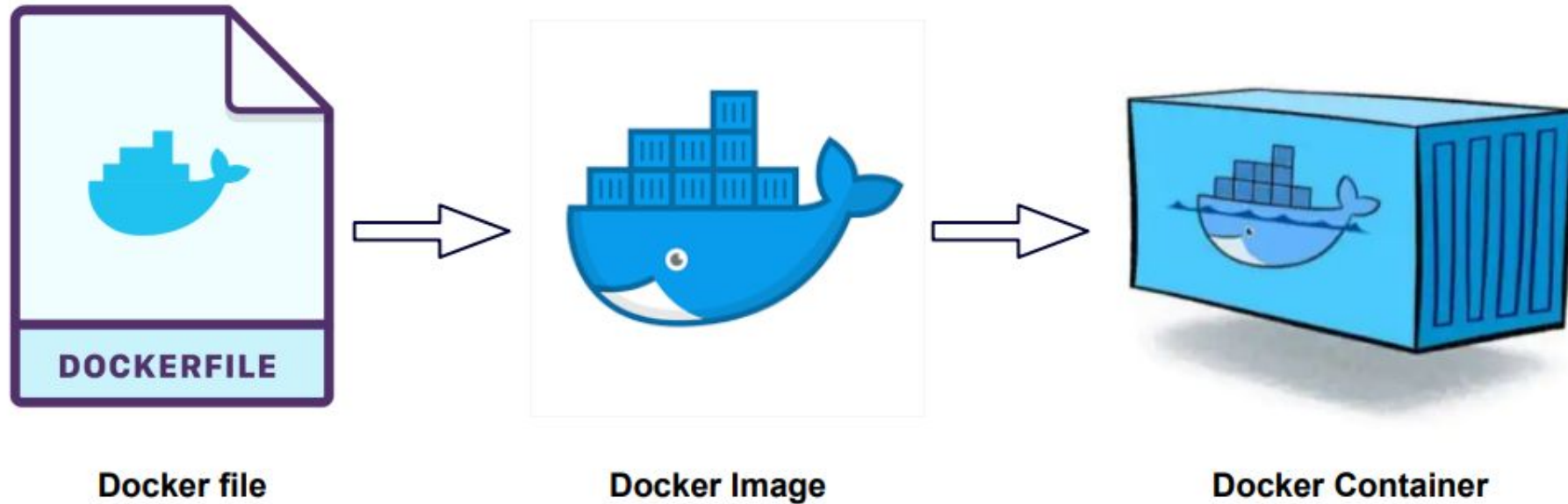
- `$ docker pull ubuntu` # Pull image to server
- `$ docker push username/my-repo` # Push image to docker hub
- `$ docker rm -f container-name` # Force to remove container
- `$ docker rmi -f image-name` # Force to remove image
- `$ docker build -t name:tag .`
“.” Build image in current directory; Default file “**Dockerfile**”
- `$ docker build -t name:tag -f /home/seng/spring.dfile .`
- `$ docker build -t name:tag -f /home/seng/spring.dfile /home/seng`
spring.dfile is Dockerfile directory; **/home/seng** is build directory
- `$ docker run -d -p 5555:5432 --name postgres-db -e POSTGRES_PASSWORD=123 postgres`
-d: detach mode; **-p**: host-port:container-port; **-e**: environment variable

Docker Command

- `$ docker images` # Show all images
- `$ docker rename old-container-name new-container-name` # rename container
- `$ docker top container-name` # Check container process
- `$ docker search image-name` # Search images from docker hub
- `$ docker restart container-name` # Stop and start container
- `$ docker rm -f $(docker ps -aq)` # Remove all kind containers
- `$ docker system prune -a` # Remove all stopped and dangling containers...
- `$ docker image prune -a` # Remove all unused and dangling images
- `$ docker volume prune` # Remove all volumes
- `$ docker cp loop.sh 59f5337d438b:/` # Copy file into container
- `$ docker cp festive_curie:/docker-entrypoint.sh` # Copy file from cont...



Dockerfile, Image and Container



Dockerfile, Image and Container

- A **Dockerfile** is a text document that contains all the commands a user could call on the command line to assemble an image.
- A **Docker image** is a file used to execute code in a Docker container. Docker images act as a set of instructions to build a Docker container, like a template. Docker images also act as the starting point when using Docker. An image is comparable to a snapshot in virtual machine (VM) environments.
- A **Docker Container** is the ready application created from Docker Image. Or you can say it is running instances of the Image and it holds the entire package needed to run the application.

Dockerfile, Image and Container

Fundamental
Instructions



FROM [baseImage]
ARG [parametre]

Configuration
Instructions



RUN [commands]
ADD | COPY [file_name]
ENV [environment_variable]

Execution
Instructions



CMD [executed_command]
ENTRYPOINT [execute_command]
EXPOSE [port_number]

Dockerfile, Image and Container

Frequently used Dockerfile commands

- **FROM** - Defines a base image, it can be pulled from docker hub (for example- if we want to create a javascript application with node as backend then we need to have node as a base image, so it can run node application.)
- **RUN** - Executes command in a new image layer(we can have multiple run commands)
- **CMD** - Command to be executed when running a container(It is asked to have one CMD command, If a Dockerfile has multiple CMDs, it only applies the instructions from the last one.
- **EXPOSE** - Documents which ports are exposed (It is only used for documentation)
- **ENV** - Sets environment variables inside the image
- **COPY** - It is used to copy your local files/directories to Docker Container.
- **ADD** - It is more feature-rich version of the COPY instruction. COPY is preferred over ADD. Major difference b/w ADD and COPY is that ADD allows you to copy from URL that is the source can be URL but in COPY it can only have local ones.

Dockerfile, Image and Container

- **ENTRYPOINT** - Define a container's executable (You cannot override and ENTRYPOINT when starting a container unless you add the --entrypoint flag.)
- **VOLUME** - It defines which directory in an image should be treated as a volume. The volume will be given a random name which can be found using docker inspect command.
- **WORKDIR** - Defines the working directory for subsequent instructions in the Dockerfile(Important point to remember that it doesn't create a new intermediate layer in Image)
- **ARG** is used to set Environment variables with key and value during the image build.
- **MAINTAINER** is used to about the person who creates the Docker Image
- **LABEL** is used to specify metadata information of Docker Image.

Dockerfile, Image and Container

- `$ sudo vim test-1.dockerfile`

1

```
FROM ubuntu
```

```
RUN apt-get update -y
```

```
RUN echo "echo "google.com"" > google.sh
```

```
RUN echo "echo "facebook.com"" > facebook.sh
```

```
ENTRYPOINT ["sh", "facebook.sh"] # Exec form
```

```
ENTRYPOINT sh google.sh # Shell form
```

2

```
FROM ubuntu
```

```
RUN apt-get update -y
```

```
RUN echo "echo "google.com"" > google.sh
```

```
RUN echo "echo "facebook.com"" > facebook.sh
```

```
CMD sh facebook.sh
```

```
CMD ["sh", "google.sh"]
```

Dockerfile, Image and Container

- `$ docker build -t test-1 -f test-1.dockerfile .`
- `$ docker run -d -it --name test-1 test-1`
- `$ docker logs test-1`
- The result will be the same for those **CMD** and **ENTRYPOINT** and only the last one will be execute.

```
seng@ip-172-31-17-136:~$ docker logs test-1  
google.com
```

Dockerfile, Image and Container

- `$ docker run -d -it --name test-1 test-1 date`
- `$ docker logs test-1`
- Pasting value from parameter won't be overridden value of **ENTRYPOINT**.

1

```
seng@ip-172-31-17-136:~$ docker run -d -it --name test-1 test-1 date
2372a82d1fcdaf7d3362456d14338ad1aa34c91eb769b469fe5aeebaa7b66dd6
seng@ip-172-31-17-136:~$ docker logs 2372a82d1fcdaf7d3362456d14338ad1aa34c9
```

Dockerfile, Image and Container

- `$ docker run -d -it --name test-2 test-2 date`
- `$ docker logs test-2`
- Pasting value from parameter will be overridden value of **CMD**.

2

```
seng@ip-172-31-17-136:~$ docker run -d -it --name test-2 test-2 date
9601ee40c26ca91c49f110c2b3c18b05d27a950f0fda72bda7999896114e9bf2
seng@ip-172-31-17-136:~$ docker logs 9601ee40c26ca91c49f110c2b3c18b05d2
Wed Nov  9 11:41:44 UTC 2022
```

Dockerfile, Image and Container

- `$ sudo vim test-3.dockerfile`

```
FROM ubuntu
```

```
RUN apt-get update -y
```

```
RUN echo "echo \"google.com\"" > google.sh
```

```
RUN echo "echo \"facebook.com\"" > facebook.sh
```

```
ENTRYPOINT sh facebook.sh
```

```
CMD sh google.sh
```

Dockerfile, Image and Container

- `$ docker build -t test-3 -f test-3.dockerfile .`
- `$ docker run -d -it --name test-3 test-3`
- **CMD** cannot be overridden the value of **ENTRYPOINT**.

```
seng@ip-172-31-17-136:~$ docker logs c7be09bfaca90d  
facebook.com/ginx
```

Dockerfile, Image and Container

- `$ sudo vim test-3.dockerfile`

```
FROM ubuntu
```

```
RUN apt-get update -y
```

```
RUN echo "echo \"google.com\"" > google.sh
```

```
RUN echo "echo \"facebook.com\"" > facebook.sh
```

```
ENTRYPOINT ["sh"]
```

```
CMD ["facebook.sh"]
```

Dockerfile, Image and Container

- `$ docker build -t test-3 -f test-3.dockerfile .`
- `$ docker run -d -it --name test-3 test-3`
- There's a special version of the exec form for **CMD** where if the first item in the array isn't a command then all items will be used as parameters for the **ENTRYPOINT**.

```
seng@ip-172-31-17-136:~$ docker logs c7be09bfaca90d
facebook.com nginx
```

- `$ docker run -it --name test-4 --entrypoint sh test-4 -c 'sh "google.sh"'`

We can override the value of **ENTRYPOINT** with **--entrypoint**

```
seng@ip-172-31-17-136:~$ docker run -it --name test-4 --entrypoint sh test-4 -c 'sh "google.sh"'
google.com
```

Dockerfile, Image and Container

- `$ sudo vim nginx.dfile`

```
ARG VERSION=1.22.1
```

```
FROM nginx:${VERSION}
```

```
ADD
```

```
https://img-19.commentcamarche.net/cl8qqj-finfDcmx6jMK6Vr-krEw=/1500x/smart/b829396acc244fd484c5ddcdcb2b08f3/ccmcms-commentcamarche/20494859.jpg ./
```

```
WORKDIR /usr/share/nginx/html
```

```
ARG NUM=1
```

```
COPY ./nginx-${NUM}/index.html ./index.html
```

```
# ./nginx-${NUM}/index.html = / not root directory in server but current context directory where we built.
```

```
ENTRYPOINT ["nginx", "-g", "daemon off;"]
```

Dockerfile, Image and Container

- `$ docker build -t nginx-2 --build-arg NUM=2 -f nginx.dfile .`
- `$ docker run -d -p 82:80 --name nginx-2 nginx-2`
- `$ docker exec -it nginx-2 bash`

```
[seng@ip-172-31-17-136:~]$ docker exec -it nginx-2 bash
[root@3b231a57b656:/usr/share/nginx/html# ls
50x.html  index.html
[root@3b231a57b656:/usr/share/nginx/html# cat index.html
SERVER 02
[root@3b231a57b656:/usr/share/nginx/html# cd /
[root@3b231a57b656:/# ls
20494859.jpg  bin  boot  dev  docker-entrypoint.d  docker-entrypoint.sh
```

Dockerfile, Image and Container

- `$ sudo vim postgres.dfile`

```
FROM postgres:13.8
```

```
ENV POSTGRES_PASSWORD=123
```

```
ENV POSTGRES_DB=test
```

```
ENV POSTGRES_USER=dara
```

- `$ docker build -t postgres-13.8 -f postgres.dfile .`
- `$ docker run -d -p 5454:5432 --name postgres-13-8 postgres-13.8`
- **OR** `$ docker run -d -p 5454:5432 --name postgres-13-8 -e POSTGRES_PASSWORD=123 -e POSTGRES_DB=test -e POSTGRES_USER=dara postgres:13.8`

Dockerfile, Image and Container

- `$ sudo vim postgres-env`

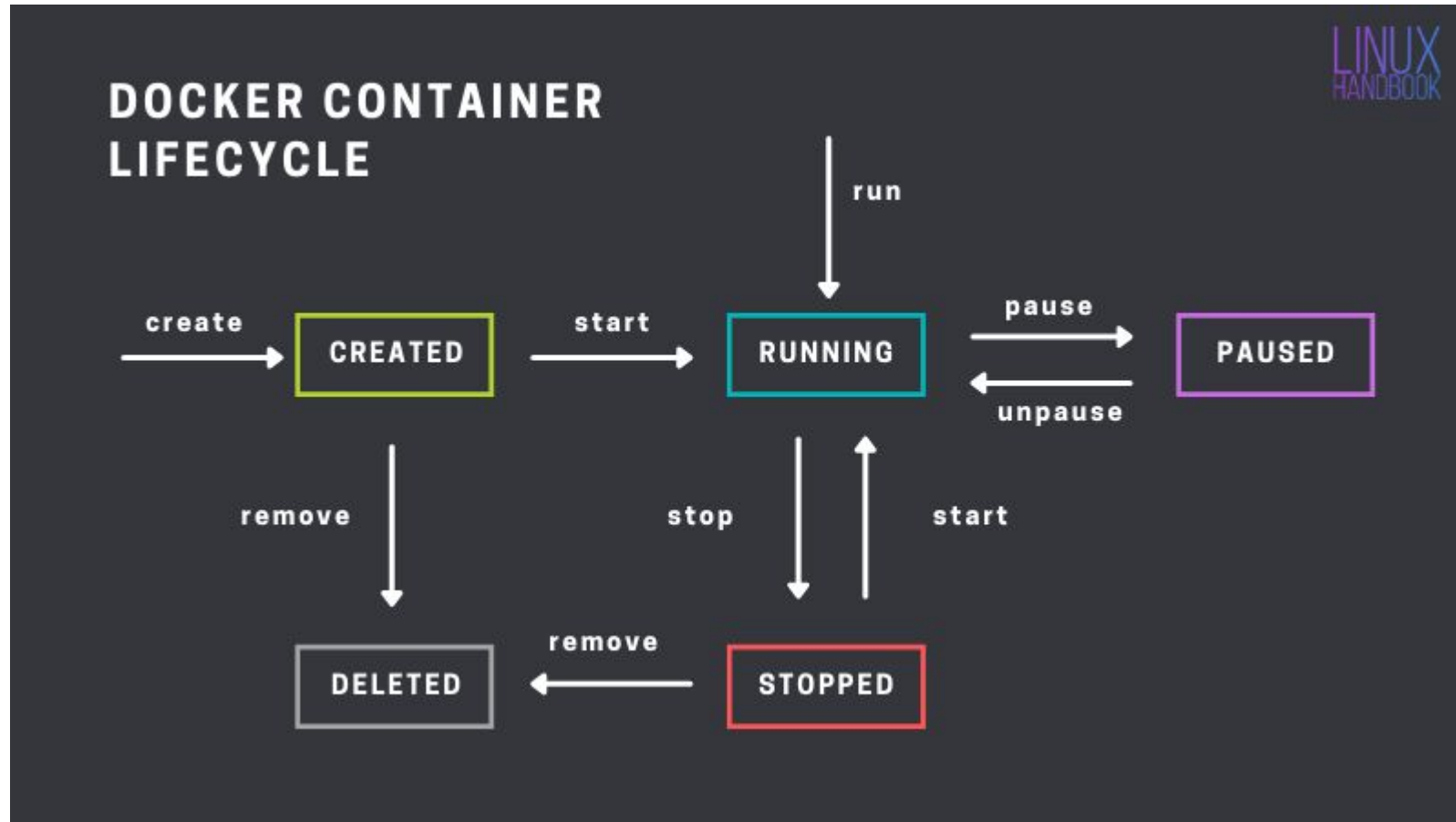
```
POSTGRES_PASSWORD=123  
POSTGRES_DB=test  
POSTGRES_USER=dara
```

- `$ docker run -d --restart always -p 5454:5432
--env-file=./postgres-env --name postgres-13-8 postgres-13.8`

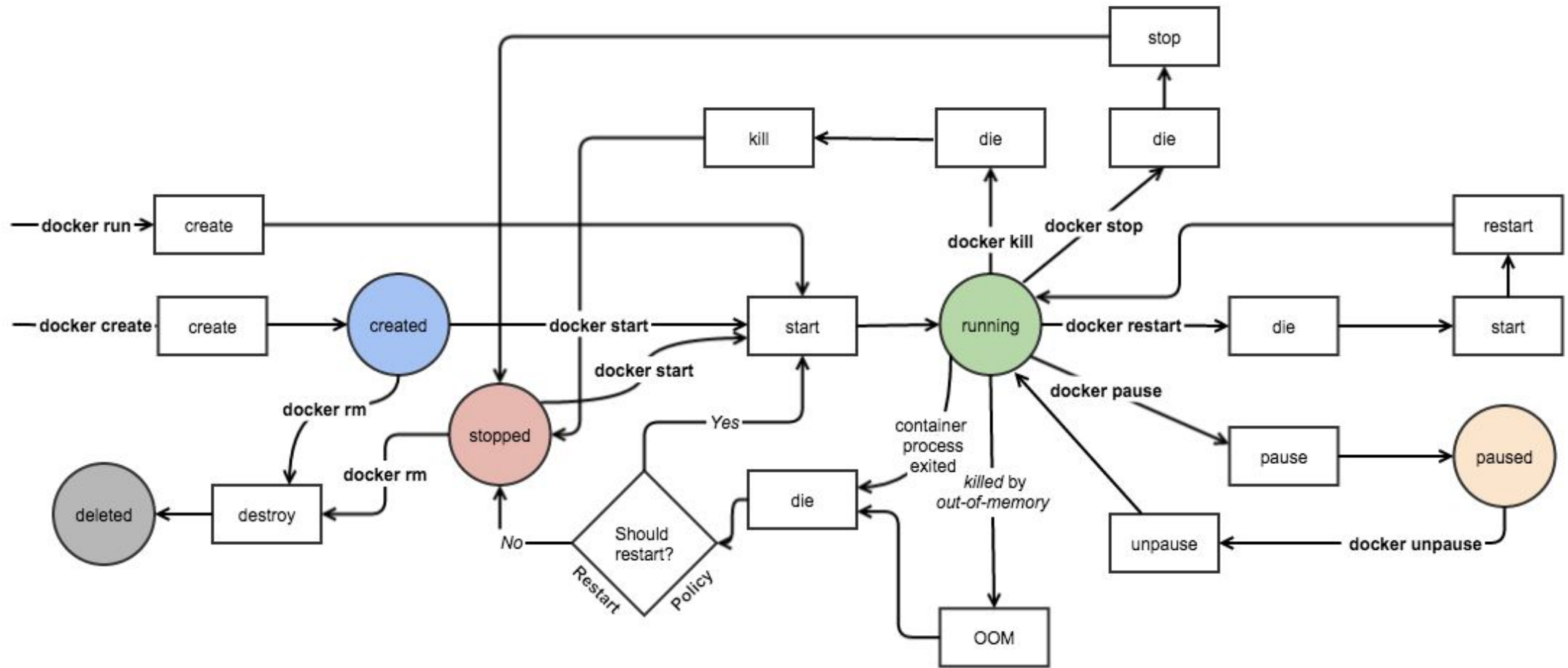
Dockerfile, Image and Container

- **no** :- it is the default value, and it will never restart the container.
- **on-failure** :- it will restart the container whenever it encounters an error, or say, whenever the process running inside the container exit with non-zero exit code. Exit code :- 0 means no error, we terminated the process intentionally, but any non-zero value is an error.
- **always** :- as the name, it will always restart the container, no matter whatever be the exit code is. Also, it will restart the container even when we manually stopped it but for that we need to restart the docker daemon.
- **unless-stopped** :- it is similar to the always flag the only difference is once the container is stopped manually it will not restart automatically even after restarting the docker daemon, until we start the container manually again.

Docker Container Lifecycle



Docker Container Lifecycle



Docker Container Lifecycle

- `$ docker create --name my-cont nginx`
 - `$ docker start my-cont`
 - `$ docker restart my-cont`
 - `$ docker run -d -p 81:80 --name my-nginx nginx:1.22.1`
- # `--name` is optional. If not it will generate a new random name.
- `$ docker pause my-nginx` # We also can use container ID instead of name
 - `$ docker unpause my-nginx`
 - `$ docker stop my-nginx`
 - `$ docker kill my-nginx`
- # kill the process, immediately. As the process cannot catch the signal and it cannot cleanup.

Docker Container Lifecycle

- `$ docker stats 75bf1bfb672e`

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
75bf1bfb672e	new-postgres	0.00%	30.72MiB / 966.2MiB	3.18%	1.44kB / 0B	3.58MB / 63.2MB	7

- `$ docker rm my-nginx`
- `$ docker rm -f my-nginx` # -f: force to remove the running container

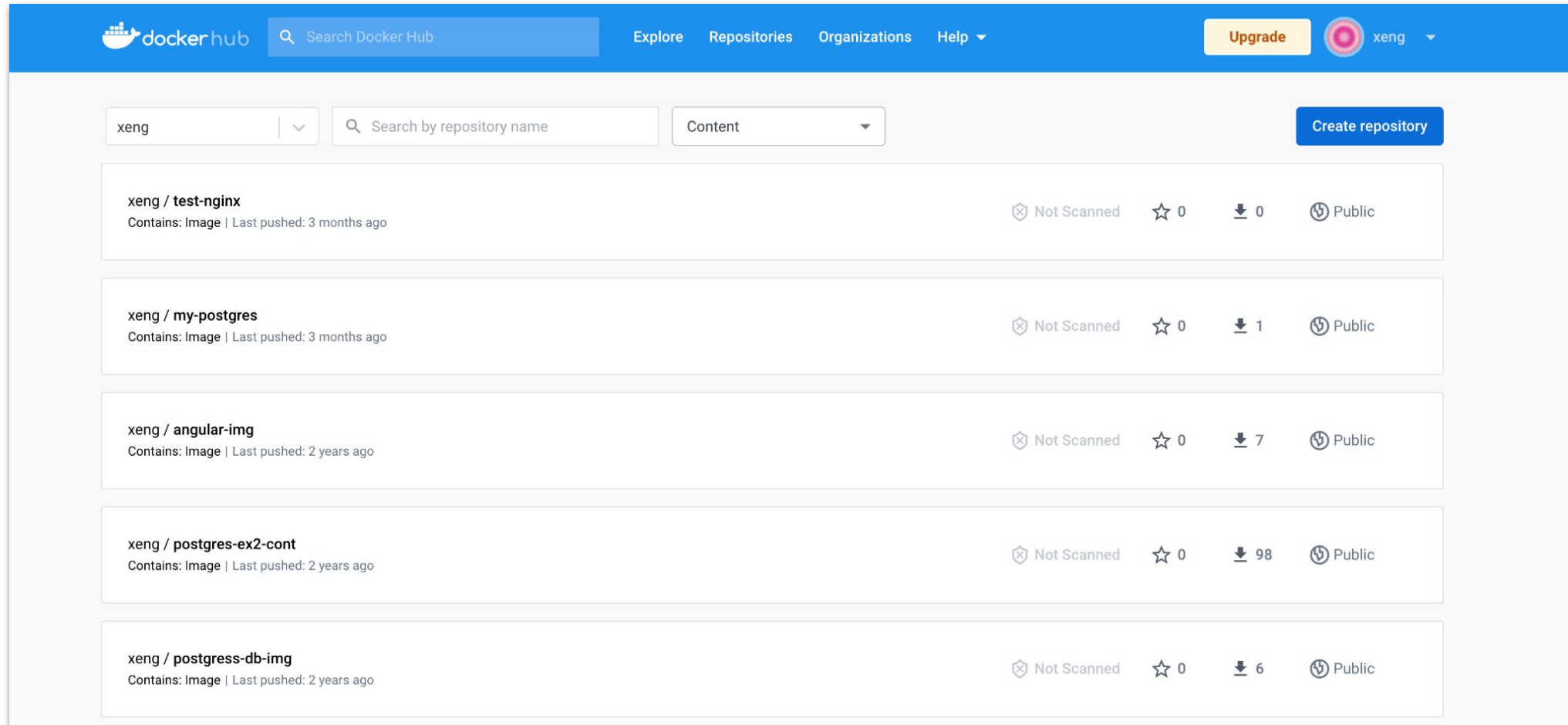
Docker Registry

- There are many ways to store images in Docker Registry
 - **Docker Hub** is a repository service and it is a cloud-based service where people push their Docker Container Images and also pull the Docker Container Images from the Docker Hub anytime or anywhere via the internet. It provides features such as you can push your images as private or public. **Other: Nexus, Gitlab...**
 - **Local Docker Registry** allows you to share your custom base images within your organization, keeping a consistent, private, and centralized source of truth for the building blocks of your architecture.

One Image = One Repository; One Repository has many tags

Docker Registry

- Docker Hub: <https://hub.docker.com/>



The screenshot shows the Docker Hub interface for a user named 'xeng'. The top navigation bar is blue with the Docker Hub logo, a search bar, and links to Explore, Repositories, Organizations, and Help. A yellow 'Upgrade' button and the user's profile icon are on the right. Below the navigation bar, there's a search bar with 'xeng' entered, a 'Search by repository name' button, and a 'Content' dropdown menu. A 'Create repository' button is on the right. The main content area lists five repositories:

Repository Name	Contains	Last pushed	Not Scanned	Stars	Downloads	Public
xeng / test-nginx	Image	3 months ago	Not Scanned	0	0	Public
xeng / my-postgres	Image	3 months ago	Not Scanned	0	1	Public
xeng / angular-img	Image	2 years ago	Not Scanned	0	7	Public
xeng / postgres-ex2-cont	Image	2 years ago	Not Scanned	0	98	Public
xeng / postgres-db-img	Image	2 years ago	Not Scanned	0	6	Public

Docker Registry

- Docker Hub: <https://hub.docker.com/>
 - `$ docker build -t xeng/postgres-custom:1.0.0 .`
`# xeng/postgres-custom:1.0.0 [username]/[repository's name]:[tag]`
 - `$ docker tag old-image-name xeng/postgres-custom:1.0.0`
`# Create new image from the existing image`
 - `$ docker login`
 - `$ docker push xeng/postgres-custom:1.0.0`
 - `$ docker pull xeng/postgres-custom:1.0.0`
 - `$ docker search nginx` # Search images by name from Docker hub

Docker Registry

- Local Docker Registry

- \$ docker run -d -p 5000:5000 --restart always --name registry registry:2
- \$ curl http://13.229.201.100:5000/v2/_catalog
- \$ docker build -t localhost:5000/custom-pg:1.1 -f postgres.dfile . --no-cache
- \$ docker push localhost:5000/custom-pg:1.1
- \$ docker run -d -p 5252:5432 --name my-db localhost:5000/custom-pg:1.1
- \$ curl http://13.229.201.100:5000/v2/custom-pg/tags/list

```
seng@ip-172-31-17-136:~$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
075cc66c1c2a	46257b2536d1	"docker-entrypoint.s..."	19 minutes ago	Up 19 minutes	0.0.0.0:5252->5432/tcp, :::5252->5432/tcp	my-db
c6333e3a7678	registry:2	"/entrypoint.sh /etc..."	32 minutes ago	Up 32 minutes	0.0.0.0:5000->5000/tcp, :::5000->5000/tcp	registry

```
seng@ip-172-31-17-136:~$ curl http://13.229.201.100:5000/v2/_catalog
{"repositories":["custom-pg"]}

seng@ip-172-31-17-136:~$ curl http://13.229.201.100:5000/v2/custom-pg/tags/list
{"name":"custom-pg","tags":["latest","1.1"]}
```

Reactjs Dockerfile

```
# Use a Node.js base image
FROM node:14 as REACT_BUILD
# Set the working directory
WORKDIR /app
# Copy the package.json and package-lock.json files
COPY package*.json ./
# Install the dependencies
RUN npm install
# Copy the rest of the application files
COPY . .
# Build the React.js application
RUN npm run build
# Use a lightweight web server such as nginx for serving the built application
FROM nginx:alpine
# Copy custom nginx configuration
COPY default.conf /etc/nginx/conf.d/default.conf
# Copy the built application from the previous step
COPY --from=REACT_BUILD /app/build /usr/share/nginx/html
# Expose port 80
EXPOSE 80
# Start the nginx server in foreground
CMD [ "nginx", "-g", "daemon off;" ]
```

Reactjs Dockerfile

```
#default.conf
server {
    listen      80;
    listen  [::]:80;
    server_name localhost;
    # Set the maximum file size that NGINX can handle to 10GB(default 1MB)
    client_max_body_size 10G;
    location / {
        root    /usr/share/nginx/html;
        index  index.html index.htm;
        #Paste this code below
        try_files $uri $uri/ /index.html;
        #rewrite  ^/reactjs/?(.*) /$1 break;
        proxy_set_header Host $http_host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

    }
    error_page   500 502 503 504  /50x.html;
    location = /50x.html {
        root    /usr/share/nginx/html;
    }
}
```

Reactjs Dockerfile

- `docker build -t reactjs-img .`
- `docker run -d -p 81:80 --name reactjs-cont reactjs-img`

A large, faint watermark of the ISTAD logo is centered in the background. It is a circular emblem with a blue outer ring containing the text 'INSTITUTE OF SCIENCE AND TECHNOLOGY ADVANCED DEVELOPMENT' and Khmer text. Inside the ring is a blue circle with a white atomic symbol. The word 'ISTAD' is written in red across the bottom of the inner circle. A blue speech bubble with three white dots is positioned over the center of the logo.

Thank you

Begin with the theory, Start with the practical